

Secuestro de DLL

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando un programa carga una DLL sin ruta absoluta, Windows la busca por un orden de directorios. Si un proceso privilegiado busca una DLL que no existe (o busca antes en un directorio donde puedes escribir), colocas ahí una DLL maliciosa con ese nombre y el proceso la carga y ejecuta tu código. Es el análogo Windows del path-hijacking de Linux.

El orden de búsqueda de DLLs incluye el directorio del ejecutable y otros; si la app no fija rutas y falta una DLL, el hueco es explotable. Tu DLL (con su DllMain) ejecuta el payload en el contexto del proceso, que puede ser SYSTEM.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando un servicio o app privilegiada tiene una DLL ausente o cargable desde un directorio escribible. ProcMon (filtrando NAME NOT FOUND en .dll) lo revela.

Cómo identificarla

- ProcMon muestra un proceso privilegiado con 'NAME NOT FOUND' buscando una .dll.
- El directorio donde busca la DLL es escribible por tu usuario.
- Una app de terceros que carga DLLs desde su propia carpeta escribible.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Filtra ProcMon por NAME NOT FOUND y Path ends with .dll mientras el proceso privilegiado corre.

Evidencia esperada: Una ruta de DLL no encontrada en un directorio donde puedes escribir; tras colocar tu DLL, ejecución en el contexto del proceso.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

`procmon.exe (filtro: Result is NAME NOT FOUND, Path ends with .dll)`

Objetivo y contexto: ProcMon monitoriza en vivo que ficheros abre cada proceso. Filtrando DLLs no encontradas ves exactamente que hueco puedes rellenar con tu DLL.

Resultado esperado: Rutas de DLLs que un proceso busca sin éxito. Si alguna está en un directorio escribible por ti y el proceso es privilegiado, ese es tu vector.

Capa 2: prueba documentada

`x86_64-w64-mingw32-gcc pe.c -shared -o legit.dll`

Objetivo y contexto: Compila (desde Kali) una DLL cuyo DllMain ejecuta tu payload al cargarse. El nombre debe coincidir con la DLL que el proceso busca.

Resultado esperado: El fichero legit.dll. Colocalo en el directorio donde el proceso la busca; al reiniciarse/cargarla, ejecuta tu código en su contexto (idealmente SYSTEM).

Anatomía de los payloads

La primera prueba es `procmon.exe` (filtro: `Result is NAME NOT FOUND, Path ends with .dll`). Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una ruta de DLL no encontrada en un directorio donde puedes escribir; tras colocar tu DLL, ejecución en el contexto del proceso.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Con ProcMon, caza DLLs que un proceso privilegiado busca y no encuentra.
- Confirma que puedes escribir en un directorio del orden de búsqueda.
- Compila una DLL con ese nombre cuyo DllMain lance tu payload.
- Colocala y dispara/espera la carga; confirma la ejecución privilegiada.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ningún proceso privilegiado busca una DLL en un directorio escribible por ti.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Una app privilegiada que busca una DLL en un directorio escribible carga la tuya: es el PATH hijacking de Windows. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(dll-hijacking\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.